Bonus Challenge - Advanced Security Implementation

1.0 Overview

This document covers the three optional bonus security challenges implemented on top of the core Week 4-6 tasks. These challenges were undertaken to demonstrate excellence in security implementation beyond the standard requirements.

Three bonus tasks were completed during the project: implementing Zero Trust Security principles for authentication and access control, deploying a Web Application Firewall (WAF) for additional protection, and conducting a Social Engineering Attack simulation with documented findings.

2.0 Zero Trust Security

2.1 What is Zero Trust?

Zero Trust is a modern security model built on the principle of "never trust, always verify". Unlike traditional security models that assume everything inside a network is safe, Zero Trust treats every request as potentially hostile - regardless of whether it originates from inside or outside the network.

The three core principles of Zero Trust are to always verify explicitly by authenticating and authorising based on all available data points, to apply least privilege by restricting user access to only what is necessary for the shortest time required, and to assume breach by minimising the attack surface, segmenting access, and ensuring end-to-end encryption is enforced.

2.2 Implementation

Zero Trust was implemented in Juice Shop by modifying the JWT (JSON Web Token) authentication system in *lib/insecurity.ts.*

2.2.1 IP-Bound Token Issuance

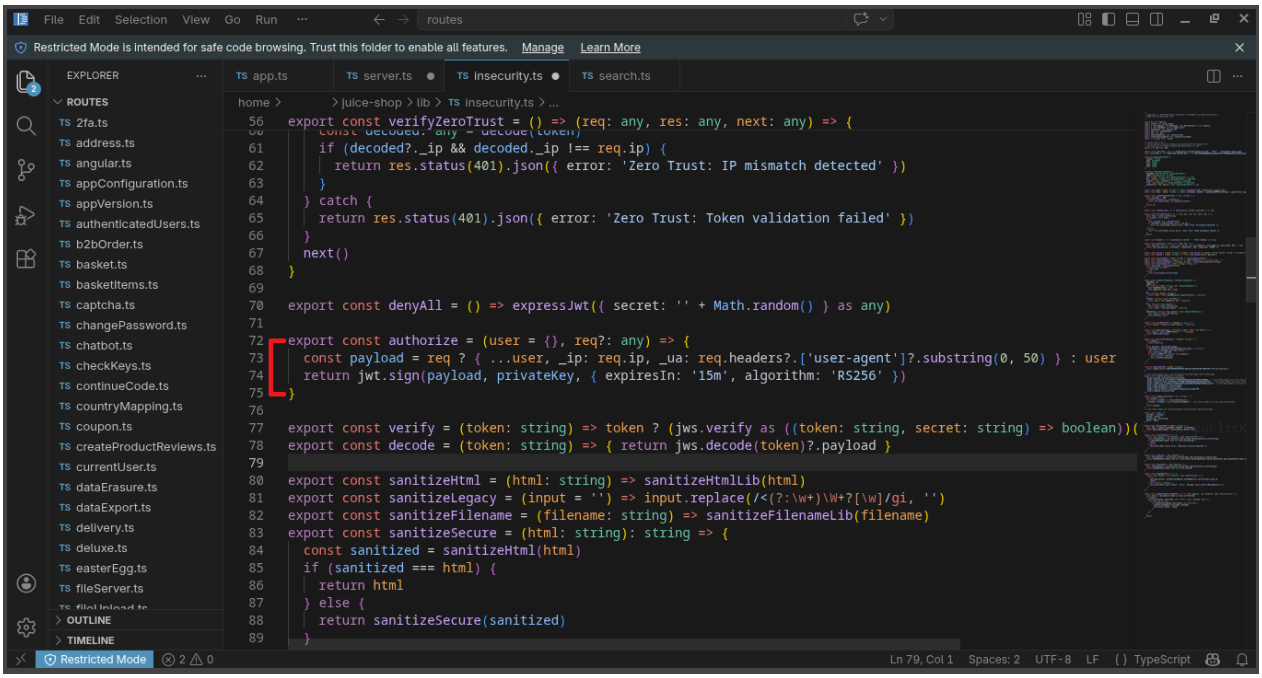
The authorize function was updated to embed the user's IP address and User-Agent into the JWT payload at the time of login

Before Implementation:

```
export const hmac = (data: string) => crypto.createHmac('sha256', 'pa4qacea4VK9t9nGv7yZtwmj').update(data).dig

export const cutOffPoisonNullByte = (str: string) => {
  const nullByte = '%00'
  if (utils.contains(str, nullByte)) {
    return str.substring(0, str.indexOf(nullByte))
  }
  return str
}

export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)

export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })

export const verify = (token: string) => token ? (jws.verify as ((token: string, secret: string) => boolean))(
export const decode = (token: string) => { return jws.decode(token)?.payload }

export const sanitizeHtml = (html: string) => sanitizeHtmlLib(html)
export const sanitizeLegacy = (input = '') => input.replace(/<(?:\w+)\W+?[\w]/gi, '')
export const sanitizeFilename = (filename: string) => sanitizeFilenameLib(filename)
export const sanitizeSecure = (html: string): string => {
  const sanitized = sanitizeHtml(html)
  if (sanitized === html) {
    return html
  } else {
    return sanitizeSecure(sanitized)
  }
}

export const authenticatedUsers: IAuthenticatedUsers = {
```

(Figure 2.2.1)

After Implementation:



```
export const verifyZeroTrust = () => (req: any, res: any, next: any) => {
    if (decoded?._ip && decoded._ip !== req.ip) {
      return res.status(401).json({ error: 'Zero Trust: IP mismatch detected' })
    }
  } catch {
    return res.status(401).json({ error: 'Zero Trust: Token validation failed' })
  }
  next()
}

export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)

export const authorize = (user = {}, req?: any) => {
  const payload = req ? { ...user, _ip: req.ip, _ua: req.headers?.['user-agent']?.substring(0, 50) } : user
  return jwt.sign(payload, privateKey, { expiresIn: '15m', algorithm: 'RS256' })
}

export const verify = (token: string) => token ? (jws.verify as ((token: string, secret: string) => boolean))(
export const decode = (token: string) => { return jws.decode(token)?.payload }

export const sanitizeHtml = (html: string) => sanitizeHtmlLib(html)
export const sanitizeLegacy = (input = '') => input.replace(/<(?:\w+)\W+?[\w]/gi, '')
export const sanitizeFilename = (filename: string) => sanitizeFilenameLib(filename)
export const sanitizeSecure = (html: string): string => {
  const sanitized = sanitizeHtml(html)
  if (sanitized === html) {
    return html
  } else {
    return sanitizeSecure(sanitized)
  }
```

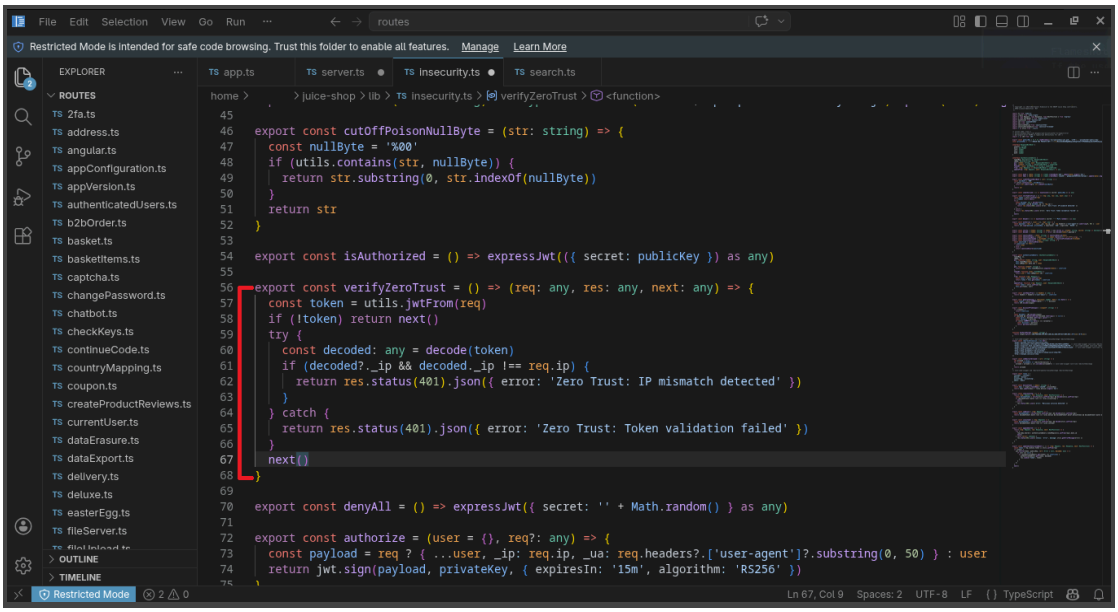
(Figure 2.2.2)

```
export const authorize = (user = {}, req?: any) => {
  const payload = req ? {
    ...user,
    _ip: req.ip,
    _ua: req.headers?.['user-agent']?.substring(0, 50)
  } : user
  return jwt.sign(payload, privateKey, { expiresIn: '15m', algorithm: 'RS256' })
}
```

Several changes were made to strengthen authentication. The JWT token expiry time was reduced from 6 hours to 15 minutes, limiting the potential impact of a stolen token. Additionally, the user's IP address (_ip) and User-Agent string (_ua) were embedded in the token payload, allowing tokens to be tied more closely to the client that generated them.

2.2.2 Zero Trust Verification Middleware

A new middleware function was added to lib/insecurity.ts that verifies the IP address on every authenticated request:



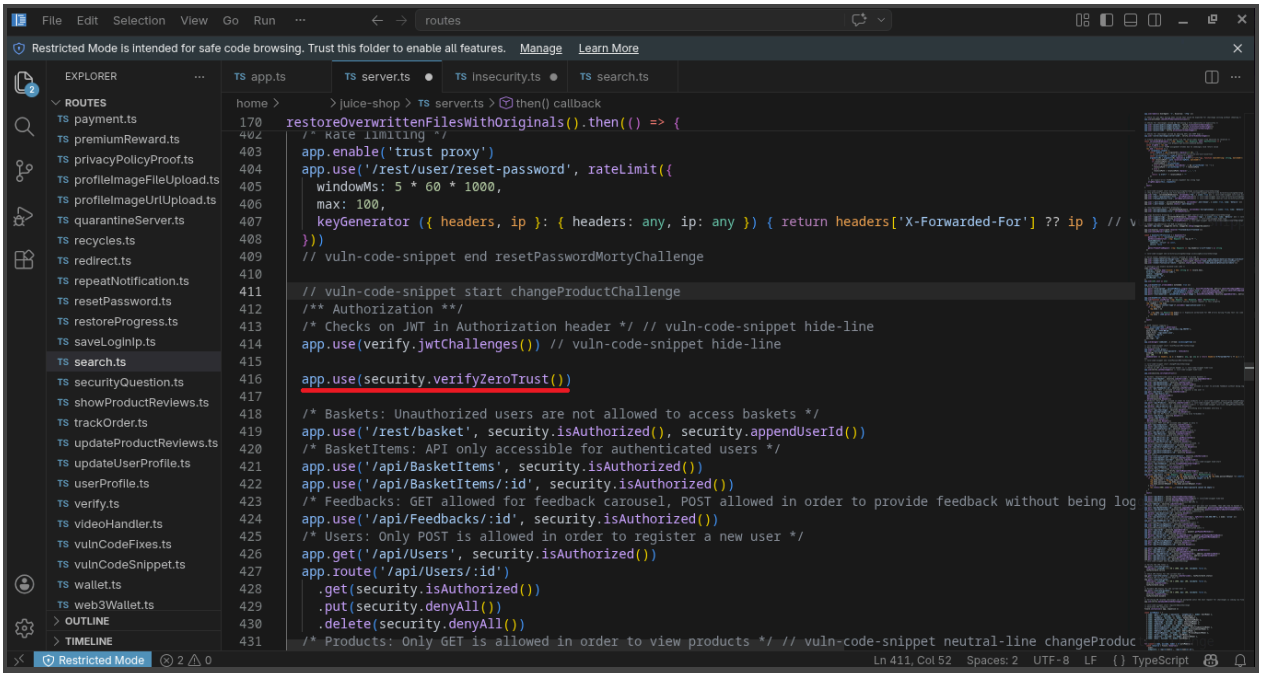
(Figure 2.2.3)

```
export const verifyZeroTrust = () => (req: any, res: any, next: any) => {
  const token = utils.jwtFrom(req)
  if (!token) return next()
  try {
    const decoded: any = decode(token)
    if (decoded?._ip && decoded._ip !== req.ip) {
      return res.status(401).json({ error: 'Zero Trust: IP mismatch detected' })
    }
  } catch {
    return res.status(401).json({ error: 'Zero Trust: Token validation failed' })
  }
  next()
}
```

2.2.3 Global Application in server.ts

The Zero Trust middleware was applied globally in server.ts after the JWT challenge verification:

app.use(verify.jwtChallenges())
app.use(security.verifyZeroTrust())



(Figure 2.2.4)

2.3 How It Works

When a user logs in, a JWT token is issued that includes their IP address and User-Agent information. For every subsequent authenticated request, the system extracts the IP address from the token and compares it with the current request's IP. If the two do not match - for example, if a stolen token is replayed from a different location - the request is immediately rejected with a 401 Unauthorized response. In addition, tokens expire after 15 minutes, significantly reducing the time window in which a compromised token can be misused.

3.0 Web Application Firewall (WAF)

3.1 What is a WAF?

A Web Application Firewall (WAF) is a security tool that monitors, filters, and blocks HTTP traffic to and from a web application. Unlike a traditional network firewall, a WAF operates at the application layer (Layer 7) and can inspect the content of HTTP requests - detecting and blocking attacks such as SQL injection, XSS, and path traversal before they reach the application logic.

3.2 Implementation

A custom WAF middleware was implemented directly in server.ts. After evaluating the express-waf package, it was found to be too aggressive for Juice Shop's Angular frontend, blocking legitimate requests. A custom implementation was chosen instead to provide precise control over what patterns are blocked.

```
// Custom WAF middleware
app.use((req: Request, res: Response, next: NextFunction) => {
  const suspiciousPatterns = [
    /<script[\s\S]*?>[\s\S]*?<\/script>/i,  // XSS - script tag injection
    /javascript:/i,                          // XSS - JavaScript protocol
    /onload\s*=/i,                           // XSS - onload event handler
    /onerror\s*=/i,                          // XSS - onerror event handler
    /eval\s*\(/i,                            // Code injection - eval execution
    /\.\.\//g                                // Path traversal - directory climbing
  ]
  const payload = JSON.stringify(req.query) + JSON.stringify(req.body)
  for (const pattern of suspiciousPatterns) {
    if (pattern.test(payload)) {
      return res.status(403).json({ error: 'WAF: Suspicious request blocked' })
```

```
    }
  }
  next()
})
```

The WAF inspects both query string parameters and request body for suspicious patterns. If a match is found, the request is rejected with 403 Forbidden before it reaches any application logic.

3.3 Attack Patterns Blocked

The pattern *<script>...</script>* is commonly associated with reflected or stored XSS attacks, where malicious scripts are injected directly into web pages. An example of this type of payload is *<script>document.cookie</script>*, which can be used to access or steal sensitive user data such as cookies.

The javascript: protocol injection pattern is used to execute JavaScript code directly through URLs. A typical example is *javascript:alert(1)*, which can trigger script execution if the application fails to properly sanitize or restrict URL inputs.

The onload= event handler pattern is used in DOM-based XSS attacks, where malicious code is executed when an element loads. For example, *<img onload=alert(1)>* can trigger JavaScript execution as soon as the image loads.

The *onerror=* event handler is another DOM XSS vector that executes code when an element fails to load correctly. An example is *<img src=x onerror=alert(1)>*, which runs the payload when the image source breaks.

The *eval(* pattern is associated with unsafe JavaScript execution, allowing arbitrary code to be run within the application. For instance, *eval(atob('bWFsd2FyZQ==))* can decode and execute encoded malicious scripts.

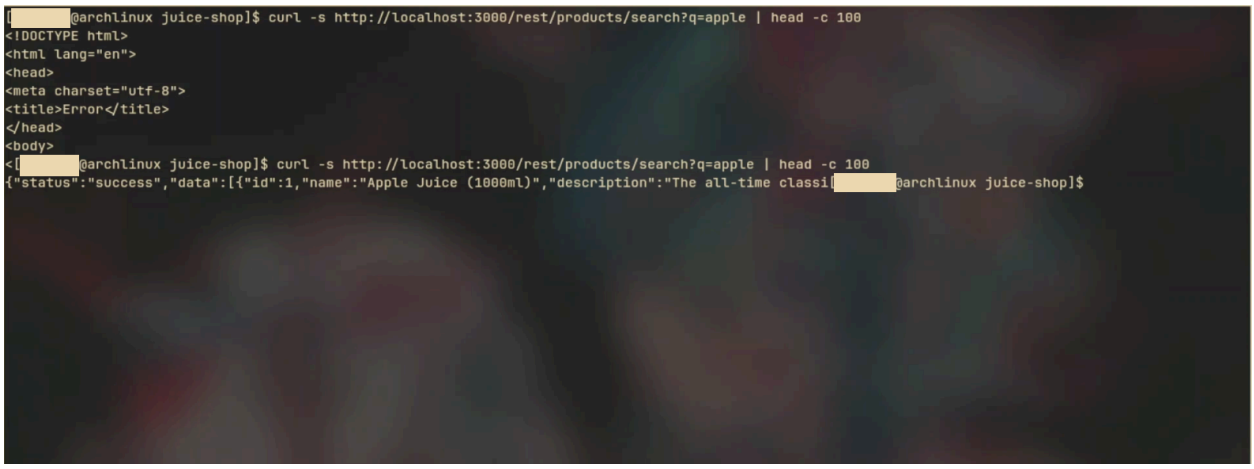
The *../* pattern is used in path traversal attacks to access restricted directories on a server. A common example is *../../etc/passwd*, which attempts to retrieve sensitive system files by navigating outside the intended directory structure.

3.4 Verification

Two tests were performed to verify the WAF:

Test 1 - Normal request (should pass):

curl -s "http://localhost:3000/rest/products/search?q=apple" | head -c 100

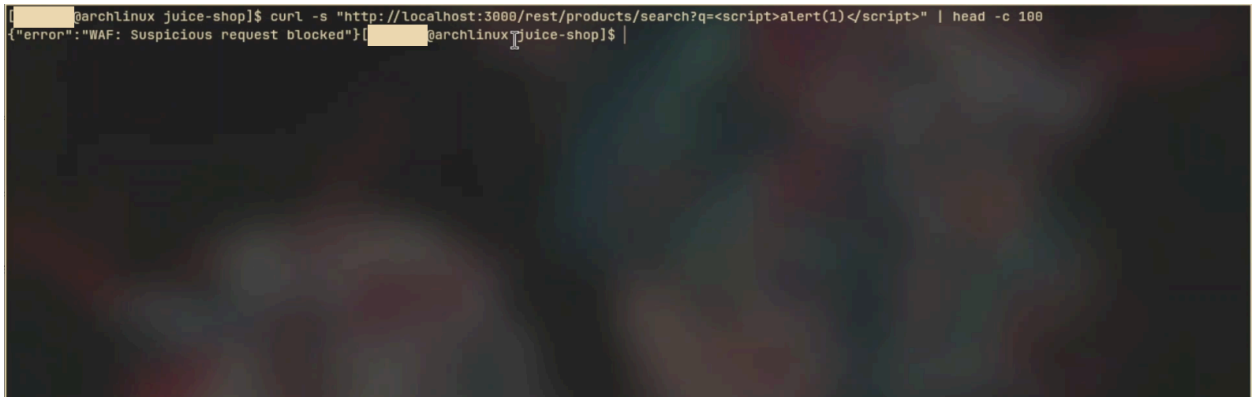


(Figure 3.4.1 - Terminal output showing WAF blocking XSS payload)

Result: {"status":"success","data":[...]} - request passed through normally

Test 2 - XSS attack (should be blocked):

curl -s "http://localhost:3000/rest/products/search?q=<script>alert(1)</script>" | head -c 100



(Figure 3.4.2 - Terminal output showing WAF blocking XSS payload)

Result: {"error":"WAF: Suspicious request blocked"} - attack was blocked

4.0 Social Engineering Simulation

4.1 What is Social Engineering?

Social engineering is the psychological manipulation of people into performing actions or revealing confidential information. Unlike technical attacks that exploit software vulnerabilities, social engineering exploits human psychology - targeting emotions such as trust, fear, urgency, and helpfulness.

Social engineering is one of the most effective attack vectors because no technical control can fully protect against it. It bypasses firewalls, encryption, and authentication systems by targeting the human element.

4.2 Simulated Attack Scenarios

Four social engineering attack scenarios were simulated against the Juice Shop environment. No real users were targeted - all scenarios are simulated for awareness and documentation purposes (Written as such in the Assignment given).

4.2.1 Scenario 1 - Phishing Email Campaign

Technique: Email impersonation with domain spoofing
Target: All registered Juice Shop users
Goal: Steal login credentials

A phishing email was simulated using a spoofed domain (juice-sh0p.com instead of juice-shop.com) and urgency tactics to trick users into clicking a malicious link:

> **From:** support@juice-sh0p.com
> **Subject:** Urgent: Your Juice Shop account has been compromised
>
> "We have detected suspicious activity on your account. Please verify your identity immediately or your account will be suspended within 24 hours."

Description: This method is effective because the fake domain closely resembles the real one (e.g., using “0” instead of “o”), making it hard to notice. It also relies on fear of account loss and urgency, which can cause users to act quickly without verifying the source.

Mitigation: Implement DMARC, SPF, and DKIM email authentication. Train users to inspect sender domains carefully.

4.2.2 Scenario 2 - Pretexting (Fake IT Support)

Technique: Authority and urgency via phone/chat impersonation
Target: System administrator
Goal: Gain admin credentials

> "Hi, this is Alex from DevOps. We're doing emergency database maintenance and need your admin credentials temporarily to complete the migration. The deployment window closes in 10 minutes."

Description: This technique is effective because it impersonates authority figures like IT or DevOps teams, making the message seem legitimate. The use of urgency pressures the user into acting quickly without verification, while technical jargon adds false credibility and makes the request appear more trustworthy.

Mitigation: Establish a firm policy that credentials are never shared, even with IT staff. Always verify identity through a second channel.

4.2.3 Scenario 3 - USB Baiting

Technique: Physical USB device planted in a public area
Target: Developer or admin workstation
Goal: Install malware or keylogger

> A USB drive labeled "Juice Shop - Q2 2026 Salary Review" is left in the office parking lot. A curious employee plugs it in, executing a malicious payload disguised as a PDF document.

Description: This method is effective because it exploits human curiosity, encouraging users to open or interact with the content. The use of an appealing or urgent label makes it seem important and relevant, increasing the likelihood of engagement. In some cases, it can also bypass network security controls if the system trusts the source or file type.

Mitigation: Disable USB autorun on all workstations. Train staff to never plug in unknown devices. Deploy endpoint protection that scans removable media.

4.2.4 Scenario 4 - Spear Phishing (Targeted Admin Attack)

Technique: Targeted phishing using OSINT gathered from the application
Target: Juice Shop administrator
Goal: Account takeover

During the technical assessment, the following information was gathered that could enable a highly targeted attack

Information: Internal information was identified from multiple sources within the application. The endpoint */rest/admin/application-configuration* exposed internal IP addresses, while the *x-recruiting* response header revealed job postings and organizational structure details. The */ftp/* directory listing exposed internal documents, and stack traces in 500 error responses disclosed file paths and underlying technology stack information.

Using this information, an attacker could craft a convincing spear phishing message:

> "Hi (Admin Name), I saw your job posting at /#/jobs. I'm a security researcher and found a critical vulnerability in your Juice Shop deployment running at 192.168.99.100:3000. I've attached a detailed report - please review urgently."
>
> *[Attachment: security-report.pdf.exe]*

Description: This type of attack is more dangerous than generic phishing because it uses real, application-specific information gathered from reconnaissance, making the message appear highly credible and legitimate. By referencing actual internal details such as an internal IP address, it further reinforces trust and reduces suspicion from the victim. Additionally, it is often targeted at high-value users with system access, increasing the potential impact if the attack is successful.

Mitigation: Remove information disclosure vulnerabilities. Restrict access to admin configuration endpoints. Train admins to be especially skeptical of unsolicited security reports.

4.4 Key Finding - Technical Vulnerabilities Enable Social Engineering

Internal IP disclosure can significantly increase the effectiveness of phishing attacks, as attackers can reference real internal addresses in emails to make them appear legitimate and trustworthy. This added realism can reduce suspicion and improve the chances of user interaction.

Exposure of FTP directory listings provides attackers with insider knowledge about available files and system structure, which can then be used to craft more convincing and targeted social engineering messages.

Stack traces shown in error responses reveal sensitive details such as file paths and the underlying technology stack, helping attackers identify potential weaknesses and tailor their exploitation techniques accordingly.

The presence of job postings in HTTP headers can also expose organizational structure and team names, which attackers can use to impersonate internal staff or departments, increasing the credibility of phishing or impersonation attempts.

4.5 Ways To Prevent Such Attacks

Email security should be strengthened by implementing DMARC, SPF, and DKIM to reduce the risk of domain spoofing and email impersonation. Regular security awareness training, including

phishing simulation exercises, should be conducted to help staff recognize and respond to social engineering attempts.

A strict credential policy should be enforced where passwords and authentication details are never shared with anyone, including IT personnel. In addition, verification procedures should require confirming identity through a separate, independent communication channel before any sensitive action is taken.

To reduce physical attack vectors, a USB policy should prohibit the use of unknown or untrusted USB devices, and autorun features should be disabled across all systems. Information disclosure risks should also be addressed by restricting or removing endpoints that expose internal system details.

Finally, a clear incident reporting process should be established so that staff can quickly report suspicious emails, messages, or interactions, enabling faster response and containment of potential security incidents.